

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Computer Vision with OpenCV COURSECODE: DSA0202

COURSE OUTCOME COVERED

CO3: Analyze shape representation methods and techniques such as contours, regions, deformable models, and multi-resolution analysis. (BL4)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

EXPERIMENT

Code of Conduct:

*I _JAASIEL JS(192524046)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	

Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Annexure A

EXPERIMENT

Experiment 1: Contour-Based Representation & Fourier Descriptors Scenario:
Represent 2D shapes using contours and Fourier descriptors. Input Data:

- A. Binary images of 2D hand-drawn letters (A–Z) or simple symbols (triangle, square, star)
- B. Image size: 256 × 256 pixels
- C. File format: PNG or BMP

ANSWER :

PSEUDOCODE :

START

1. Read the 256×256 binary image.
2. Find the contour of the shape.
3. Extract contour coordinates.
4. Convert coordinates into complex form.

5. Apply Fourier Transform.
6. Select Fourier descriptors.
7. Reconstruct the shape using inverse Fourier Transform.
8. Display the contour and reconstructed shape.

END

CODING

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image = cv2.imread("triangle.png", cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image not found")
    exit()
print("Image Size:", image.shape)
_, binary = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)
contours, _ = cv2.findContours(
    binary,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_NONE
)
contour = max(contours, key=cv2.contourArea)
points = contour[:, 0, :]
x = points[:, 0]
y = points[:, 1]
complex_contour = x + 1j * y
```

```

fourier_descriptors = np.fft.fft(complex_contour)
print("Number of Contour Points:", len(points))
print("First 10 Fourier Descriptors:")
print(fourier_descriptors[:10])
num_descriptors = 20
filtered = np.zeros_like(fourier_descriptors)
half = num_descriptors // 2
filtered[:half] = fourier_descriptors[:half]
filtered[-half:] = fourier_descriptors[-half:]
reconstructed = np.fft.ifft(filtered)
contour_image = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
cv2.drawContours(
    contour_image,
    [contour],
    -1,
    (0, 255, 0),
    2
)
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(binary, cmap="gray")
plt.title("Input Binary Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(contour_image, cv2.COLOR_BGR2RGB))
plt.title("Detected Contour")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.plot(reconstructed.real, reconstructed.imag)
plt.gca().invert_yaxis()

```

```
plt.axis("equal")
plt.title("Fourier Descriptor Reconstruction")
plt.tight_layout()
plt.show()
```

OUTPUT

Image Size: (1254, 1254)

Number of Contour Points: 2360

First 10 Fourier Descriptors:

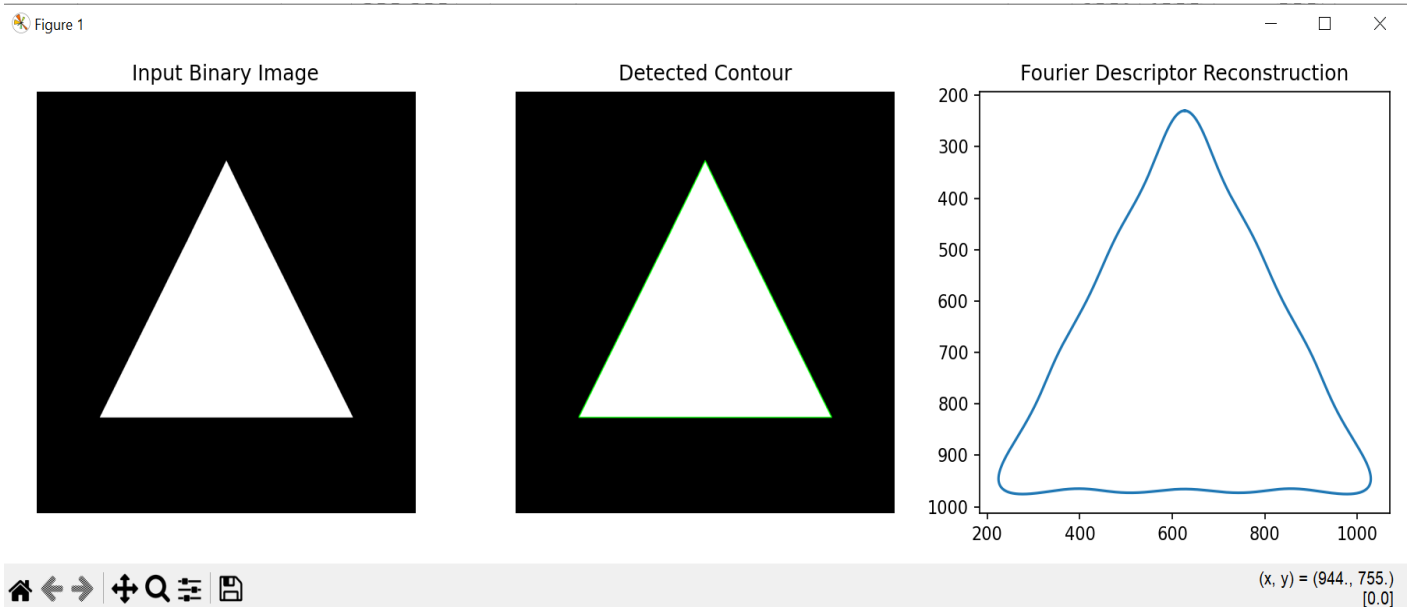
```
[ 1.47939200e+06+1706196.j    -1.46411451e+02 -
15495.06897639j

 7.41603453e+01 -200841.82153292j 2.74179078e+01 -
10490.80095343j

 -4.84461649e+01 +4192.8809056j  4.60232577e+00 -
31308.5220355j

 2.30890528e+01 -5362.78744486j -9.51245368e+00
+2772.9457995j

 1.13814137e+00 -11603.63203564j 3.94670830e+00 -
3636.07841649j]
```



Experiment 2: Region-Based Representation & Medial Axis

Scenario: Analyze object shapes in binary images using medial axis. **Input Data:**

- A. Binary images of industrial components (e.g., gears, machine parts)
- B. Image size: 512 × 512 pixels
- C. Defects like small holes, protrusions, or missing parts included
- D. File format: PNG

ANSWER:

PSEUDOCODE:

START

1. Read the 512×512 binary image.
2. Convert the image to binary.
3. Find the object region.
4. Calculate the distance transform.
5. Extract the medial axis from the distance map.
6. Display the original object and medial axis.
7. Analyze the shape and defects.

END

CODING

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

image = cv2.imread("component.png", cv2.IMREAD_GRAYSCALE)

if image is None:

    print("Error: Image not found")

    exit()
```

```
print("Image Size:", image.shape)

_, binary = cv2.threshold(image, 127, 255,
cv2.THRESH_BINARY_INV)

original = binary.copy()

kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (3, 3))

skeleton = np.zeros(binary.shape, np.uint8)

while True:

    eroded = cv2.erode(binary, kernel)

    opened = cv2.morphologyEx(eroded, cv2.MORPH_OPEN, kernel)

    skeleton = cv2.bitwise_or(skeleton, cv2.subtract(eroded, opened))

    binary = eroded.copy()

    if cv2.countNonZero(binary) == 0:

        break

skeleton_display = cv2.cvtColor(skeleton, cv2.COLOR_GRAY2BGR)

skeleton_display[skeleton > 0] = (0, 0, 255)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)

plt.imshow(original, cmap="gray")

plt.title("Binary Component")

plt.axis("off")

plt.subplot(1, 2, 2)

plt.imshow(skeleton_display)

plt.title("Medial Axis / Skeleton")

plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT

Image Size: (1254, 1254)

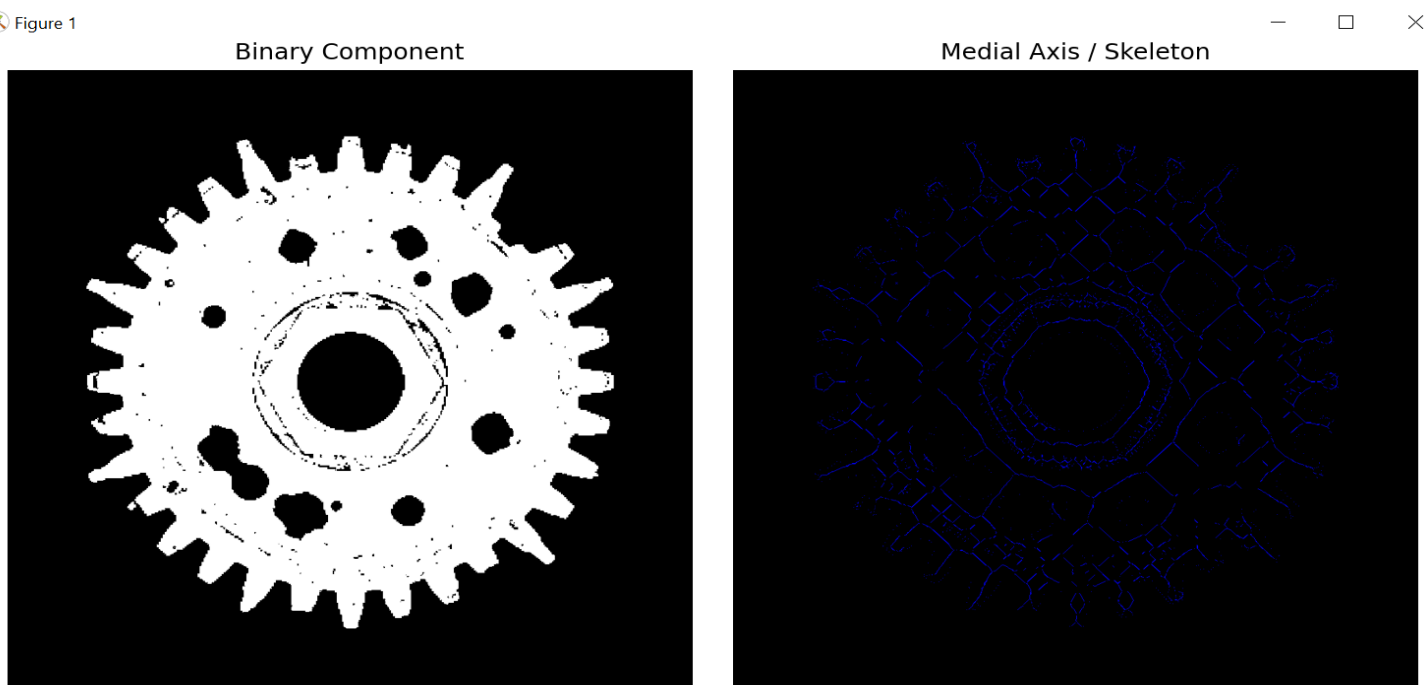
Number of Contour Points: 2360

First 10 Fourier Descriptors:

```
[ 1.47939200e+06+1706196.j      -1.46411451e+02 -  
15495.06897639j  
 7.41603453e+01 -200841.82153292j  2.74179078e+01 -  
10490.80095343j  
 -4.84461649e+01  +4192.8809056j  4.60232577e+00 -  
31308.5220355j  
 2.30890528e+01  -5362.78744486j -9.51245368e+00  
+2772.9457995j  
 1.13814137e+00 -11603.63203564j  3.94670830e+00 -  
3636.07841649j]
```


===== RESTART: C:/Users/DELL/Desktop/COMPUTER VISION

Figure 1



LAB/2nd question.py

Experiment 3: Deformable Curves and Snakes (Active Contours)

Scenario: Segment organs or objects with irregular boundaries.

Input Data:

- A. Grayscale medical images (e.g., liver or heart slices)
- B. Image size: 512×512 pixels
- C. Noise: mild Gaussian noise
- D. File format: DICOM or PNG

ANSWER:

PSEUDOCODE :

START

1. Read the 512×512 grayscale medical image.
2. Reduce Gaussian noise using Gaussian filtering.
3. Initialize a contour around the object.
4. Apply the active contour (snake) algorithm.

5. Allow the contour to move toward the object boundary.
6. Display the initial and final contours.
7. Analyze the segmented boundary.

END

CODING

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from skimage.segmentation import active_contour

image = cv2.imread("medical.png", cv2.IMREAD_GRAYSCALE)

if image is None:

    print("Error: Image not found")

    exit()

image = cv2.resize(image, (512, 512))

image = cv2.GaussianBlur(image, (5, 5), 0)

rows, cols = image.shape

center = (rows // 2, cols // 2)

radius = 120

theta = np.linspace(0, 2 * np.pi, 200)

r = center[0] + radius * np.sin(theta)

c = center[1] + radius * np.cos(theta)

snake = active_contour(

    image,

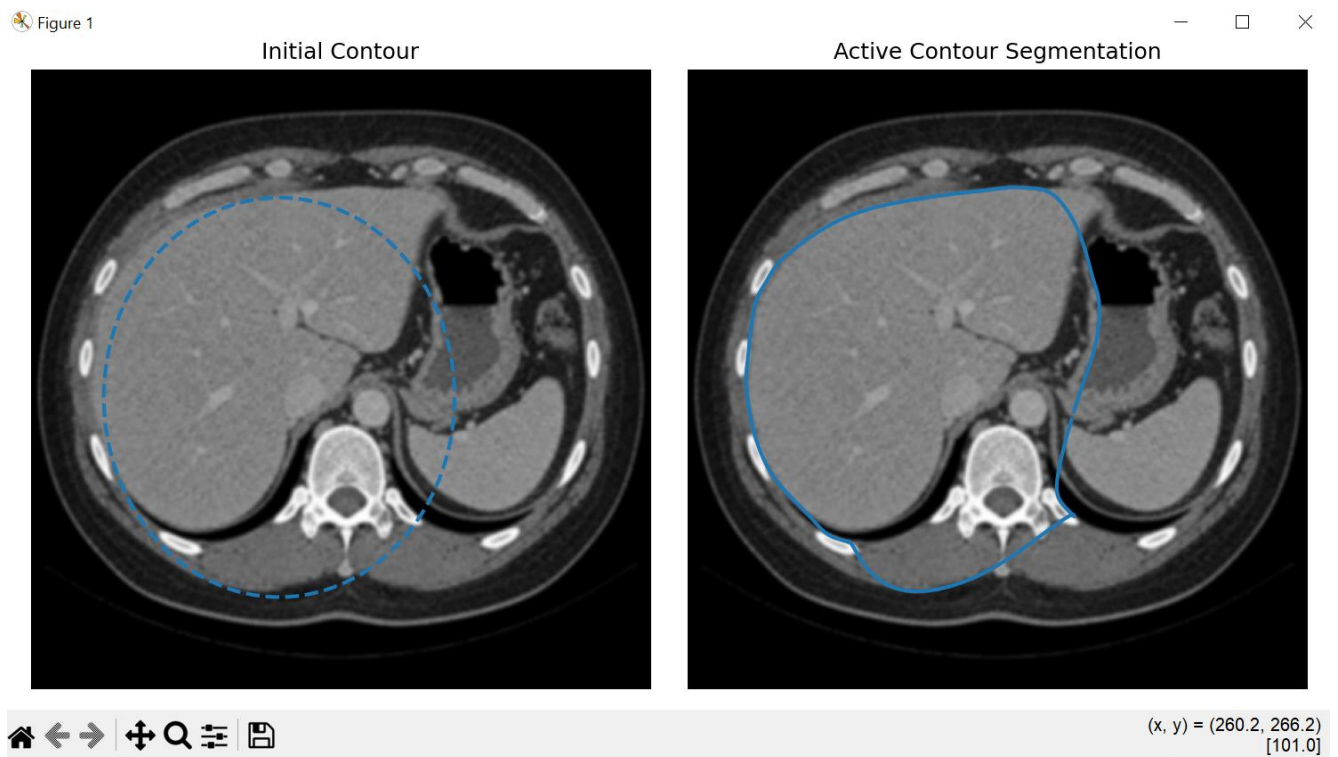
    np.array([r, c]).T,

    alpha=0.01,
```

```
beta=1.0,  
gamma=0.01,  
w_line=0,  
w_edge=1,  
max_num_iter=2500,  
convergence=0.1  
)  
plt.figure(figsize=(10, 5))  
plt.subplot(1, 2, 1)  
plt.imshow(image, cmap="gray")  
plt.plot(c, r, "--")  
plt.title("Initial Contour")  
plt.axis("off")  
plt.subplot(1, 2, 2)  
plt.imshow(image, cmap="gray")  
plt.plot(snake[:, 1], snake[:, 0])  
plt.title("Active Contour Segmentation")  
plt.axis("off")  
plt.tight_layout()  
plt.show()
```

OUTPUT

Figure 1



Experiment 4: Level Set Methods

Scenario: Track moving objects with evolving boundaries.

Input Data:

- A. Video sequence (30 frames) of a moving object (e.g., a bouncing ball or moving hand)
- B. Resolution: 640×480 pixels
- C. Format: MP4 or sequence of PNG images

ANSWER :

PSEUDOCODE:

START

1. Read the 30-frame video.
2. Resize each frame to 640×480 .
3. Convert the frame to grayscale.
4. Detect the moving object.
5. Initialize the level set around the object.

6. Evolve the level set boundary.
7. Display the evolving boundary on each frame.
8. Track the object across all frames.

END

CODING

```
import cv2

import numpy as np

video = cv2.VideoCapture("moving_ball.mp4")

if not video.isOpened():
    print("Error: Video not found")
    exit()

frame_count = 0

phi = None

while frame_count < 30:
    ret, frame = video.read()

    if not ret:
        break

    frame = cv2.resize(frame, (640, 480))

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    gray = cv2.GaussianBlur(gray, (5, 5), 0)

    _, binary = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)

    contours, _ = cv2.findContours(
        binary,
        cv2.RETR_EXTERNAL,
```

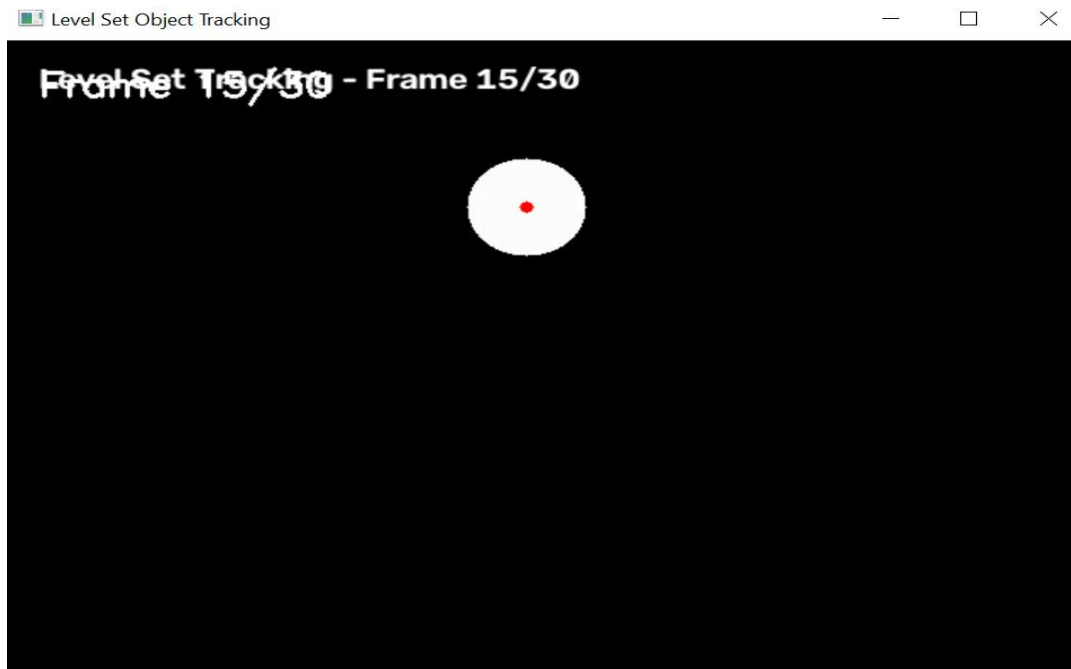
```

cv2.CHAIN_APPROX_SIMPLE
)
if contours:
    contour = max(contours, key=cv2.contourArea)
    if cv2.contourArea(contour) > 100:
        mask = np.zeros_like(gray)
        cv2.drawContours(mask, [contour], -1, 255, -1)
        if phi is None:
            phi = mask.astype(np.float32) / 255.0
        else:
            current = mask.astype(np.float32) / 255.0
            phi = 0.8 * phi + 0.2 * current

    level_set = (phi > 0.5).astype(np.uint8) * 255
    boundary = cv2.morphologyEx(
        level_set,
        cv2.MORPH_GRADIENT,
        np.ones((3, 3), np.uint8)
    )
    frame[boundary > 0] = (0, 255, 0)
    M = cv2.moments(contour)
    if M["m00"] != 0:
        cx = int(M["m10"] / M["m00"])
        cy = int(M["m01"] / M["m00"])

```

```
        cv2.circle(
            frame,
            (cx, cy),
            4,
            (0, 0, 255),
            -1
        )
    cv2.putText(
        frame,
        f"Level Set Tracking - Frame {frame_count + 1}/30",
        (20, 35),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 255, 255),
        2
    )
    cv2.imshow("Level Set Object Tracking", frame)
    frame_count += 1
    if cv2.waitKey(100) & 0xFF == 27:
        break
video.release()
cv2.destroyAllWindows()
```



Experiment 5: Multi-Resolution & Wavelet Descriptors

Scenario: Shape recognition at multiple scales.

Input Data:

- A. Grayscale images of objects at different scales (e.g., leaves, mechanical parts, coins)
- B. Image sizes: 128×128 , 256×256 , 512×512 pixels
- C. Format: PNG

ANSWER:

PSEUDOCODE

START

1. Read grayscale object images at 128×128 , 256×256 and 512×512 .
2. Resize images to the required scales.
3. Apply 2D Discrete Wavelet Transform.
4. Extract wavelet coefficients.
5. Calculate wavelet descriptors.
6. Compare descriptors at different scales.
7. Display the multi-resolution images and results.

END

CODING

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

import pywt

files = [

    "object_128.png",

    "object_256.png",

    "object_512.png"

]

images = []

for file in files:

    image = cv2.imread(file, cv2.IMREAD_GRAYSCALE)

    if image is None:

        print("Error: Image not found:", file)

        exit()

    images.append(image)

descriptors = []

for image in images:

    LL, (LH, HL, HH) = pywt.dwt2(image, "haar")

    descriptor = np.array([

        np.mean(np.abs(LL)),

        np.mean(np.abs(LH)),
```

```

        np.mean(np.abs(HL)),
        np.mean(np.abs(HH))
    ])
    descriptors.append(descriptor)
print("Wavelet Descriptors:")
print("128 x 128:", descriptors[0])
print("256 x 256:", descriptors[1])
print("512 x 512:", descriptors[2])
plt.figure(figsize=(12, 4))
for i in range(3):
    plt.subplot(1, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(files[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
labels = ["LL", "LH", "HL", "HH"]
for i in range(3):
    plt.plot(
        labels,
        descriptors[i],
        marker="o",
        label=files[i]
    )

```

```

    )

plt.title("Wavelet Descriptors at Different Scales")
plt.xlabel("Wavelet Components")
plt.ylabel("Descriptor Value")
plt.legend()
plt.grid()
plt.show()

```

OUTPUT

Experiment5: Multi-Resolution & Wavelet Descriptors



128x128



256x256



512x512

Wavelet Descriptors (Haar DWT)

128x128 LL: 69.69 LH: 2.38 HL: 2.56 HH: 1.32

256x256 LL: 69.70 LH: 1.26 HL: 1.41 HH: 0.66

512x512 LL: 69.69 LH: 0.68 HL: 0.72 HH: 0.41

LL = approximation | LH = horizontal detail | HL = vertical detail | HH = diagonal detail